

南開大學
Nankai University

计算机学院

并行程序设计

ANN 选题并行优化期末报告

姓名：赵宇哲

学号：2411094

专业：计算机科学与技术

2026 年 7 月 3 日

项目仓库：https://github.com/Aurora1094/NKU-ParallelProgramming

摘　要

本文围绕 DEEP100K 数据集上的近似最近邻搜索（Approximate Nearest Neighbor Search, ANN）任务，系统研究不同索引结构与并行架构下的查询优化方法。实验从无索引 Flat Scan 出发，依次讨论 SIMD 向量化、SQ/PQ 量化粗排、IVF 倒排索引、IVF-PQ 压缩索引以及 HNSW 图索引，并进一步分析这些方法在多线程、MPI 分布式并行和 CUDA GPU 批量查询场景下的性能表现。

综合结果表明，ANN 查询的性能瓶颈会随着算法结构和并行架构变化而迁移：Flat Scan 阶段主要受距离计算开销限制，SIMD 和量化方法能够降低单次距离计算与粗排成本；IVF 通过候选簇裁剪减少扫描规模，IVF-PQ 进一步降低存储和近似距离估计开销，但需要 rerank 控制召回损失；HNSW 通过图导航在高召回场景下取得较低查询延迟。并行架构方面，SIMD 主要优化底层距离计算，多线程提高 query 级吞吐，MPI 通过数据分片降低本地搜索规模，CUDA 则适合批量矩阵计算、GPU Top-K 和访问局部性优化。

本文最终从 Recall-Latency 权衡、计算/访存/通信瓶颈迁移以及不同应用场景下的方法选择三个角度，对 ANN 并行优化进行了统一总结。实验说明，高性能 ANN 查询并不是单纯依赖某一种索引或硬件加速方式，而是需要结合候选生成、距离估计、结果合并和硬件执行特点进行整体设计。

关键词：近似最近邻搜索；ANN；SIMD；多线程；MPI；CUDA；IVF；IVF-PQ；HNSW

目录

1 前言 **1**
1.1 实验环境 . . . 1
1.2 实验内容概述 . . . 1
1.2.1 ANN 算法背景 . . . 1
1.2.2 ANNS 的优化方向 . . . 2
1.2.3 本文综合实验路线 . . . 2
1.3 数据集与评价指标 . . . 3

2 无索引扫描与量化方法 **3**
2.1 Flat Scan 基线方法 . . . 3
2.2 Flat-SIMD 向量化优化 . . . 4
2.3 SQ-SIMD 粗排与精排 . . . 4
2.4 PQ-ADC 与 rerank 方法 . . . 5
2.5 实验结果与分析 . . . 5

3 IVF 类索引的并行化 **6**
3.1 IVF 基本原理 . . . 6
3.2 数据划分与索引构建策略 . . . 7
3.2.1 先构建全局 IVF 再分发索引 . . . 7
3.2.2 先划分原始数据再构建本地 IVF . . . 7
3.2.3 本文实验采用的策略 . . . 7
3.3 单机多线程 IVF . . . 8
3.3.1 无多线程 baseline . . . 8
3.3.2 Pthread 查询级并行 . . . 8
3.3.3 OpenMP 查询级并行 . . . 8
3.3.4 线程调度与负载均衡 . . . 9
3.4 MPI-IVF 分布式并行 . . . 9
3.4.1 rank 间数据分片 . . . 9
3.4.2 query broadcast 与 batch 通信 . . . 9
3.4.3 local top-k 搜索 . . . 10
3.4.4 global top-k 合并 . . . 10
3.4.5 通信开销与负载均衡 . . . 10
3.5 GPU-IVF 批量查询 . . . 10
3.5.1 centroid 搜索 . . . 11
3.5.2 batch 内候选簇搜索 . . . 11
3.5.3 query grouping 与访问局部性 . . . 11
3.6 实验结果与分析 . . . 12

4 IVF-PQ 类索引的并行化 **12**
4.1 IVF-PQ 基本原理 . . . 12
4.2 PQ 码本训练与向量编码 . . . 13

4.3 PQ 查表粗排与 rerank 13
4.4 单机多线程 IVF-PQ 14
4.4.1 PQ LUT 构建并行 14
4.4.2 候选粗排并行 14
4.4.3 rerank 精排阶段 14
4.5 MPI-IVF-PQ 分布式并行 15
4.5.1 rank 本地 PQ 编码 15
4.5.2 local approximate search 15
4.5.3 rerank 与 global merge 15
4.6 GPU IVF-PQ-lite 16
4.6.1 压缩表示与存储开销 16
4.6.2 GPU 查询流程 17
4.6.3 Recall 损失来源 17
4.7 实验结果与分析 17

5 HNSW 类图索引的并行化 18
5.1 HNSW 基本原理 18
5.2 单机多线程 HNSW 18
5.2.1 多 query 并行 19
5.2.2 efSearch 对查询性能的影响 19
5.3 MPI shard-HNSW 19
5.3.1 数据分片与本地图构建 20
5.3.2 local HNSW 搜索 20
5.3.3 全局候选合并 20
5.4 IVF-HNSW 混合索引 21
5.4.1 簇内 HNSW 构建 21
5.4.2 nprobe 与 efSearch 的共同影响 21
5.5 centroid-HNSW + IVF-HNSW 21
5.6 实验结果与分析 22

6 跨方法综合讨论 22
6.1 不同索引的 Recall-Latency 特征 23
6.2 计算、访存与通信瓶颈迁移 23
6.3 SIMD、多核、MPI 与 CUDA 的作用差异 24
6.4 不同应用场景下的方法选择 24
6.5 本次综合实验新增分析总结 25

7 总结与展望 26
7.1 实验结论 26
7.2 不足与改进方向 26

1 前言

1.1 实验环境

本文围绕近似最近邻搜索（Approximate Nearest Neighbor Search, ANN）展开综合并行化实验，实验覆盖 SIMD、多线程、MPI 分布式并行和 CUDA GPU 并行等多种并行架构。由于不同并行方式依赖的硬件环境不同，本文实验涉及 ARM 服务器、x86 本地平台和 GPU 平台三类环境。

表 1: 实验平台与软件环境

项目	ARM 平台	x86 平台	GPU 平台
操作系统	openEuler 22.03 LTS SP4	WSL Ubuntu 22.04	Windows 11
CPU 架构	aarch64	x86_64	x86_64
CPU 型号	HiSilicon Kunpeng-920	Intel Core i9-14900HX	Intel Core i9-14900HX
CPU 数量	8	32 logical CPUs	24 cores / 32 logical processors
SIMD	ARM NEON / ASIMD, 128-bit	AVX2 / FMA	AVX2 / FMA
编译器	g++ 10.3.1 / mpic++	g++ 11.4.0	CUDA 12.4 / nvcc / VS2022
并行框架	SIMD / OpenMP / Pthread / MPI	OpenMP / Pthread / SIMD	CUDA / cuBLAS
GPU	–	–	RTX 4060 Laptop GPU
编译参数	`-O2 -fopenmp -lpthread -std=c++11`	`-O2/-O3 -march=native`	`nvcc` release build

需要说明的是，不同平台在 CPU 架构、SIMD 指令集、缓存层次、内存带宽、GPU 功耗策略和编译器后端上均存在差异。因此，本文不将不同平台的绝对 latency 简单视为完全公平的硬件横向比较，而是重点分析不同算法结构和并行方式如何改变 ANN 查询中的性能瓶颈。

1.2 实验内容概述

本文以高维向量近似最近邻搜索为统一任务，围绕 Recall 与 Latency 的权衡关系，系统研究不同索引结构和并行架构对查询性能的影响。整体上，本文的实验内容可以概括为三条主线。

第一条主线是底层距离计算优化。对于最直接的 Flat Scan 方法，查询需要遍历全部 base 向量并计算高维内积，计算量较大。因此本文首先研究 SIMD 向量化、循环展开、预取和量化表示等方法，分析如何降低单次距离计算和粗排阶段的开销。

第二条主线是候选生成方式优化。仅优化距离计算仍然无法避免全量扫描，因此本文进一步引入 IVF、IVF-PQ 和 HNSW 等索引结构。IVF 通过倒排表减少候选数量，IVF-PQ 在此基础上使用压缩编码降低候选距离估计成本，HNSW 则通过图结构进行导航式搜索。这些方法从算法结构上改变了 ANN 查询访问候选的方式。

第三条主线是并行架构优化。本文分别在 SIMD、多线程、MPI 和 CUDA 架构下实现或分析 ANN 查询过程。SIMD 主要作用于单次距离计算，多线程主要利用 query-level parallelism，MPI 通过数据分片扩展到分布式搜索，CUDA 则通过 batch GEMM、GPU Top-K 和查询分组发挥 GPU 的高吞吐优势。

1.2.1 ANN 算法背景

给定 base 向量集合

$$X = \{x_1, x_2, \ldots, x_N\}, \quad x_i \in \mathbb{R}^d,$$

以及查询向量

$$q \in \mathbb{R}^d,$$

最近邻搜索的目标是在 X 中找到与 q 最相似的 top-k 个向量。在精确搜索中，算法通常需要计算 query 与所有 base 向量之间的距离或相似度；而在近似最近邻搜索中，算法允许一定误差，以换取更低的查询延迟。

本文采用内积相似度作为主要距离度量之一，其形式为

$$s(q, x_i) = \sum_{j=1}^{d} q_j x_{i,j}.$$

对于大规模高维向量集合，直接计算所有 $s(q, x_i)$ 的代价较高。因此，ANN 的核心问题并不是单纯追求最高 Recall 或最低 latency，而是在两者之间取得合适平衡。

一个典型 ANN 查询流程可以表示为：

$$\text{Query} \rightarrow \text{候选生成} \rightarrow \text{距离计算/近似距离} \rightarrow \text{top-}k \rightarrow \text{rerank} \rightarrow \text{merge} \rightarrow \text{Recall/Latency}$$

不同算法的区别主要体现在候选生成方式、距离估计方式以及 top-k 维护方式上。

1.2.2 ANNS 的优化方向

围绕上述查询流程，本文将 ANNS 的优化方向概括为四类。

第一类是计算优化。Flat Scan、rerank 和部分索引内部仍然需要大量距离计算，因此可以使用 SIMD、循环展开、多累加器和 GPU GEMM 等方式提高计算吞吐。

第二类是表示优化。SQ 和 PQ 等量化方法将 float32 向量转换为低位宽表示，从而降低存储开销和粗排计算成本。但量化会引入近似误差，因此通常需要 corrected score 或 rerank 来恢复召回率。

第三类是索引优化。IVF 通过聚类减少候选集合，IVF-PQ 结合倒排索引和压缩编码，HNSW 通过图结构进行导航式搜索。这些方法的共同目标是减少需要访问和比较的向量数量。

第四类是并行优化。多线程利用 query 之间的独立性提升吞吐；MPI 将 base 数据划分到不同 rank，降低单个进程的搜索规模；CUDA 将批量距离计算和 Top-K 选择放到 GPU 上执行，减少 CPU 后处理和数据搬运开销。

1.2.3 本文综合实验路线

本文不按单次实验编号组织内容，而是按照 ANN 方法本身的递进关系展开：先从无索引扫描出发，分析距离计算瓶颈；再引入量化粗排和 rerank，降低候选评估成本；随后使用 IVF、IVF-PQ 和 HNSW 改变候选生成方式；最后讨论这些方法在 SIMD、多线程、MPI 和 CUDA 架构下的并行化差异。

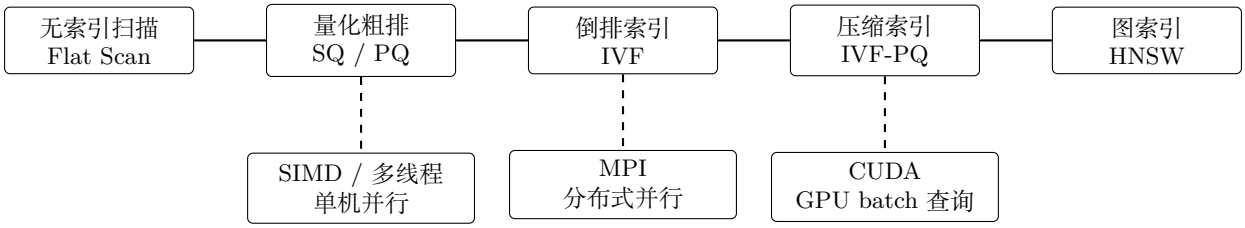


图 1: 本文综合实验路线

如图 1 所示，本文的核心逻辑是：算法结构逐步从全量扫描走向索引化搜索，并行架构逐步从单指令数据并行扩展到多线程、分布式和 GPU batch 查询。随着方法不断变化，性能瓶颈也从距离计算逐步迁移到候选访问、负载均衡、通信合并和 GPU 数据搬运。

1.3 数据集与评价指标

本文统一使用 DEEP100K 数据集。该数据集包含 100000 个 96 维 float32 base 向量、10000 个 96 维 float32 query 向量以及对应的 ground truth 近邻结果。实验任务是在 base 集合中为每个 query 返回 top-k 近似最近邻，并与 ground truth 进行比较。

本文主要使用以下评价指标：

1. **Recall@10**：衡量算法返回的前 10 个结果与 ground truth top-10 的重合程度，用于评价搜索准确率。
2. **Average Latency**：单个 query 的平均查询延迟，单位为 $\mu s/query$，用于评价在线查询效率。
3. **Speedup**：优化方法相对于 baseline 的加速比，定义为

$$Speedup = \frac{T_{\text{baseline}}}{T_{\text{optimized}}}.$$

4. **Parallel Efficiency**：用于评价多进程或多线程扩展性，定义为

$$Efficiency = \frac{Speedup}{p},$$

其中 p 表示线程数或进程数。

本文重点关注在线查询阶段性能。数据读取、索引构建、PQ 码本训练和 HNSW 建图等离线开销在需要时单独讨论，不直接计入 average latency。对于不同算法，Recall 和 latency 的含义保持一致：Recall 越高说明搜索结果越接近真实近邻，latency 越低说明单次查询越快。

2 无索引扫描与量化方法

本节讨论不依赖显式索引结构的 ANN 查询方法，包括原始 Flat Scan、SIMD 向量化扫描、SQ 量化粗排以及 PQ-ADC 近似距离计算。它们的共同特点是：候选生成仍然主要来自对 base 向量集合的扫描，只是在距离计算方式、数据表示形式和 rerank 策略上进行优化。因此，本节重点回答的问题是：在不改变候选生成结构的前提下，如何降低高维向量距离计算和粗排筛选的开销。

2.1 Flat Scan 基线方法

Flat Scan 是最直接的最近邻搜索方法。对于每个查询向量 q，算法遍历全部 base 向量 x_i，计算相似度并维护 top-k 结果。以内积相似度为例，其计算形式为

$$s(q, x_i) = \sum_{j=1}^{d} q_j x_{i,j}.$$

在 DEEP100K 数据集中，base 集合包含 100000 个 96 维 float32 向量。若对每个 query 都完整扫描所有 base 向量，则单次查询需要执行大量浮点乘加运算，时间复杂度为

$$O(Nd).$$

Flat Scan 不引入近似误差，因此 Recall 可以保持在最高水平，但其查询延迟较高。它在本文中主要作为无索引方法的精确 baseline，同时也作为后续量化粗排之后 rerank 阶段的精确计算基础。

2.2 Flat-SIMD 向量化优化

Flat-SIMD 的目标是在不改变算法结果的前提下，加速 Flat Scan 中的 float 内积计算。由于向量不同维度之间的乘加操作相互独立，适合使用 SIMD 指令进行数据级并行。

该方法主要包含以下优化：

1. 使用 SIMD 寄存器同时处理多个 float 元素；
2. 通过循环展开减少循环控制开销；
3. 使用多个累加器降低数据相关带来的流水线停顿；
4. 使用 prefetch 改善连续扫描 base 向量时的访存行为。

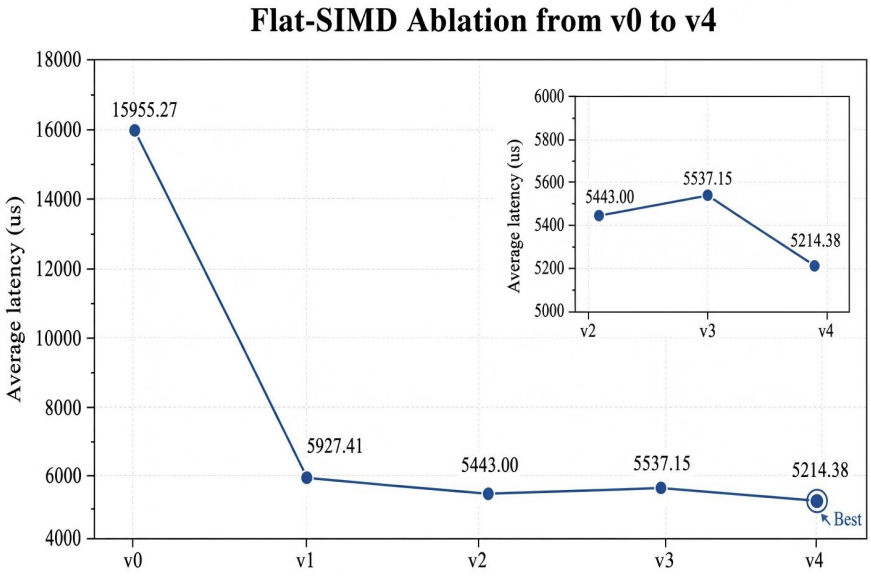


图 2: Flat-SIMD 不同优化版本的消融结果

从图 2 可以看出，随着 SIMD、循环展开、多累加器和预取优化逐步加入，查询延迟持续下降。代表性 Flat-SIMD 版本的平均查询延迟由原始 Flat Scan 的 15955.27 $\mu s/query$ 降低到 5214.38 $\mu s/query$，同时不改变原始 float 内积结果，因此 Recall 基本保持不变。这说明在无索引扫描场景中，距离计算本身是最直接的性能瓶颈。

2.3 SQ-SIMD 粗排与精排

Flat-SIMD 虽然降低了 float 内积计算成本，但仍然需要扫描全部 base 向量。为了进一步降低粗排阶段的计算和访存开销，可以使用标量量化（Scalar Quantization, SQ）将 float32 向量近似表示为

uint8 向量。

SQ-SIMD 采用“两阶段查询”：

1. 使用 uint8 SIMD 对全部 base 向量进行快速粗排；
2. 选取粗排得分最高的 top-p 个候选；
3. 对候选回到原始 float 空间进行 SIMD 精排；
4. 根据精排结果输出最终 top-k。

需要注意的是，直接使用 uint8 dot product 会破坏原始 float 内积的排序关系。因此，粗排阶段需要引入量化偏移修正，使量化得分尽量接近原始内积排序。随后通过 rerank 对候选重新计算精确 float 分数，从而恢复最终召回率。

代表性 SQ-SIMD 方法在保持 Recall@10 约 0.99995 的情况下，将平均查询延迟降低到 3675.09 $\mu s/query$。这说明量化本身并不是最终目的，更重要的是通过低位宽表示降低粗排成本，并用精排控制召回损失。

2.4 PQ-ADC 与 rerank 方法

乘积量化（Product Quantization, PQ）进一步压缩向量表示。PQ 将原始向量划分为多个子空间，并在每个子空间中训练码本。base 向量不再直接存储完整 float 向量，而是存储各子空间对应的 codeword 编号。

查询时，先为 query 构建查找表：

$$LUT_m[c] = dist(q^{(m)}, C_m[c]),$$

其中 $q^{(m)}$ 表示 query 在第 m 个子空间中的子向量，$C_m[c]$ 表示该子空间第 c 个码字。随后，候选向量的近似距离可以通过查表累加得到：

$$\widetilde{dist}(q, x_i) = \sum_{m=1}^{M} LUT_m[code_i^{(m)}].$$

PQ-ADC 的优势在于显著降低粗排阶段的存储访问和距离计算成本，但其近似误差通常大于 SQ。因此，PQ-ADC 同样需要配合 rerank：先用 PQ 近似距离筛选 top-p 候选，再对候选执行原始 float 精排。

实验结果表明，PQ-ADC 可以获得更低延迟，但 Recall 对候选数 p 更敏感。例如代表性 PQ-ADC 配置的平均延迟可降低到 1379.89 $\mu s/query$，但 Recall@10 约为 0.919541；当增大 rerank 候选数后，Recall 可以提升，但延迟也会随之上升。

2.5 实验结果与分析

表 2 汇总了无索引扫描与量化方法的代表结果。

表 2: 无索引扫描与量化方法的代表结果

方法	主要思想	Latency ($\mu s/query$)	Recall@10
Flat Scan	原始 float 全量扫描	15955.27	0.99995
Flat-SIMD	SIMD、循环展开与预取优化	5214.38	0.99995
SQ-SIMD + rerank	uint8 粗排 + float 精排	3675.09	0.99995
PQ-ADC + rerank	PQ 查表近似距离 + 精排	1379.89	0.919541
PQ-LUT + rerank	PQ 查表优化 + 更大候选精排	2033.98	0.956053

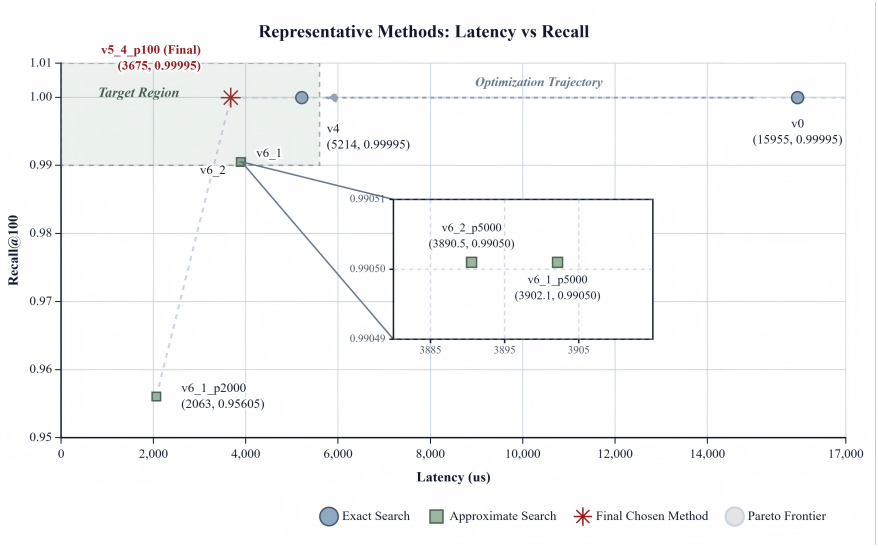


图 3: 无索引扫描与量化方法的 Recall-Latency 对比

从表 2 和图 3 可以看出，这一类方法形成了清晰的递进关系。Flat Scan 准确率最高但延迟最大；Flat-SIMD 在不损失 Recall 的情况下降低了距离计算开销；SQ-SIMD 进一步通过低位宽粗排减少计算量，并依靠 rerank 保持高召回；PQ-ADC 则提供更激进的压缩和近似搜索能力，但需要在 Recall 和 Latency 之间做更明显的权衡。

因此，本节的核心结论是：当候选生成方式仍然接近全量扫描时，优化重点首先是降低单次距离计算成本；当 SIMD 已经充分利用后，继续降低延迟需要通过量化和粗排减少进入精确计算的候选数量。这也自然引出后续索引方法：若要进一步降低延迟，仅优化距离计算还不够，需要从候选生成结构本身减少需要访问的向量规模。

3 IVF 类索引的并行化

IVF（Inverted File）类索引的核心思想是通过聚类将 base 向量划分到多个倒排表中，查询时只访问与 query 最接近的少量候选簇，从而避免全量扫描。与上一节的无索引方法相比，IVF 不再只优化单次距离计算，而是从候选生成结构上减少需要访问和比较的向量数量。因此，IVF 是本文从“计算优化”转向“索引结构优化”的关键节点。

在并行化方面，IVF 具有较好的可扩展性：在单机上，不同 query 可以由不同线程并行处理；在 MPI 环境中，base 向量可以按 rank 分片，各进程构建本地 IVF 并返回 local top-k；在 GPU 上，centroid search 和候选簇搜索可以组织成 batch GEMM，同时通过 query grouping 提高访问局部性。

3.1 IVF 基本原理

IVF 首先对 base 向量集合进行聚类，得到 $nlist$ 个聚类中心

$$C = \{c_1, c_2, \ldots, c_{nlist}\}.$$

每个 base 向量 x_i 被分配到最近的中心对应的倒排表中：

$$list(x_i) = \arg\min_j dist(x_i, c_j).$$

查询时，算法先计算 query q 与所有聚类中心之间的距离，并选出最近的 *nprobe* 个中心。随后只扫描这些中心对应的倒排表，而不是扫描全部 base 向量。其流程可以表示为：

$$q \rightarrow centroid\ search \rightarrow top\text{-}nprobe\ lists \rightarrow candidate\ scan \rightarrow top\text{-}k.$$

IVF 的性能主要由 *nlist* 和 *nprobe* 控制。*nlist* 越大，单个倒排表平均长度越短，但中心搜索开销和索引构建复杂度增加；*nprobe* 越大，访问的倒排表越多，Recall 通常越高，但 latency 也随之增加。因此，IVF 的核心是通过 *nlist* 和 *nprobe* 在候选覆盖率与候选扫描量之间取得平衡。

3.2 数据划分与索引构建策略

IVF 在不同并行架构下的索引构建策略并不相同。单机场景中，所有 base 向量可以共享同一个全局 IVF 索引；而在 MPI 分布式场景中，base 数据分布在不同进程上，需要决定是先构建全局索引再分发，还是先划分数据再构建本地索引。这个选择会影响内存占用、通信开销和负载均衡。

3.2.1 先构建全局 IVF 再分发索引

一种思路是先在单个进程或单机环境中使用完整 base 集合构建全局 IVF 索引，然后将不同倒排表或向量分发给多个进程。该方案的优点是所有数据共享同一套聚类中心，索引结构更接近完整全局 IVF，理论上更容易保持搜索一致性。

但该方案也有明显代价。首先，全局索引构建需要集中持有完整 base 数据和倒排表，内存压力较大；其次，构建完成后需要将倒排表内容分发到各 rank，会引入额外通信和数据重排开销；最后，不同倒排表长度可能差异明显，若直接按 list 分配，容易造成 rank 间负载不均。

因此，先构建全局 IVF 再分发索引更适合数据规模较小、全局索引构建成本可接受，并且希望严格复用统一聚类中心的场景。

3.2.2 先划分原始数据再构建本地 IVF

另一种策略是先按照并行单元划分原始 base 数据，再由每个并行单元在本地数据上构建 IVF 索引。对于 MPI 场景，设进程数为 P，则 base 集合被划分为

$$X = X_0 \cup X_1 \cup \cdots \cup X_{P-1}, \quad X_r \cap X_s = \varnothing.$$

每个 rank r 只负责本地分片 X_r，并构建本地 IVF 索引 I_r。

该方案更符合分布式内存模型。每个 rank 只需要保存本地数据和本地索引，索引构建阶段不需要集中处理完整 base 集合，也不需要分发完整倒排表。查询时，各 rank 对同一个 query 独立搜索本地 IVF，得到 local top-k，最后由 root 进程合并为 global top-k。

其不足在于，每个本地 IVF 只反映局部数据分布。最终结果依赖所有 rank 的 local top-k 是否覆盖真实近邻，因此需要合理设置本地搜索参数，并通过全局合并恢复最终结果。

3.2.3 本文实验采用的策略

本文采用“先划分原始数据，再构建本地 IVF”的策略来组织分布式 IVF。该策略与 MPI 的数据分片模型自然一致，也便于将 IVF 查询流程拆分为 local search 和 global merge 两部分。

整体流程如下：

1. 将 base 向量按照 rank 进行分片；
2. 每个 rank 在本地分片上构建 IVF 索引；
3. 查询阶段，root 广播 query 或 query batch；
4. 每个 rank 在本地 IVF 中执行 centroid search 和 candidate scan；
5. 各 rank 返回 local top-k；
6. root 合并所有候选，得到 global top-k。

这种设计使得 IVF 的主要并行性来自数据分片和本地搜索并行，同时也引入通信、同步和全局合并开销。后续的 MPI-IVF 分析正是围绕这些开销展开。

3.3 单机多线程 IVF

在单机环境中，IVF 的多线程并行主要采用 query-level parallelism。不同 query 之间互不依赖，因此可以由不同线程独立处理。由于 IVF 索引在查询阶段只读，多个线程可以共享同一个索引结构，不需要对倒排表加锁。

3.3.1 无多线程 baseline

无多线程 baseline 按顺序处理 query。对于每个 query，先搜索最近的 *nprobe* 个 centroid，再扫描对应倒排表中的候选向量，最后维护 top-k。该版本用于观察 IVF 本身的参数敏感性，并作为 Pthread 和 OpenMP 并行版本的对照。

IVF 的单线程性能主要取决于 *nlist* 和 *nprobe*。如果 *nprobe* 太小，搜索范围不足，Recall 会下降；如果 *nprobe* 太大，候选扫描量增加，latency 上升。因此，baseline 的意义不仅是提供时间基准，也用于确定合适的索引参数范围。

3.3.2 Pthread 查询级并行

Pthread 版本通过手动创建线程并划分 query 区间实现并行。每个线程处理一部分 query，并独立执行完整 IVF 搜索流程。该方法的优点是线程划分明确，调度开销可控；缺点是代码复杂度较高，且静态划分 query 时可能出现负载不均。

IVF 中不同 query 访问的倒排表长度可能不同，因此每个 query 的候选扫描量并不完全一致。如果某些线程分到的 query 更“重”，就会拖慢整体 wall latency。Pthread 的性能因此不仅取决于线程数，还取决于 query 分配方式和倒排表长度分布。

3.3.3 OpenMP 查询级并行

OpenMP 版本通常通过 `parallel for` 并行化 query 循环。与 Pthread 相比，OpenMP 实现更简洁，也更容易切换线程数和调度策略。在相同索引参数下，OpenMP 不改变单个 query 的 IVF 搜索过程，因此 Recall 应与串行版本保持一致。

OpenMP 的关键在于 schedule 策略。对于每个 query 计算量相近的算法，静态调度已经足够；但对于 IVF，不同 query 访问的倒排表长度可能不同，动态调度有时能够缓解负载不均。不过，动态调度也会引入额外运行时开销，因此需要在任务粒度和调度成本之间折中。

3.3.4 线程调度与负载均衡

IVF 的线程负载不均主要来自候选数量差异。即使所有线程处理相同数量的 query，不同 query 的实际扫描候选数也可能不同。因此，多线程 IVF 的优化重点不只是增加线程数，还包括合理设置 schedule、batch 粒度和任务划分方式。

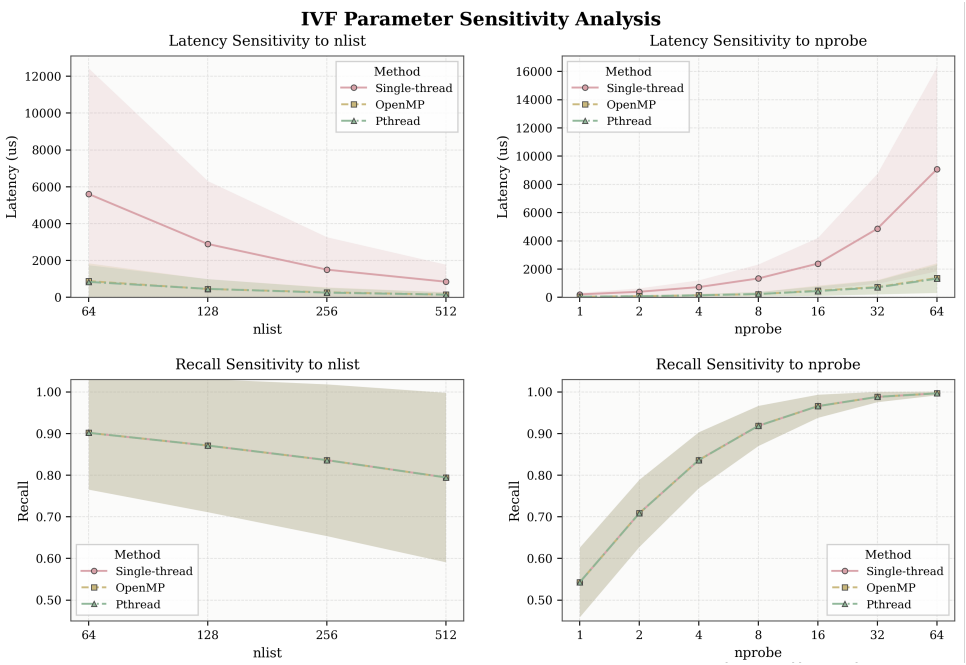


图 4: IVF 参数对 Recall 与 Latency 的影响

图 4 展示了 IVF 参数对 Recall 和 latency 的影响。可以看到，*nprobe* 增大能够提升 Recall，但也会增加候选扫描量；*nlist* 增大可以缩短单个倒排表长度，但也可能使小 *nprobe* 下的候选覆盖不足。因此，多线程 IVF 的调优需要同时考虑索引参数和线程调度。

3.4 MPI-IVF 分布式并行

在 MPI 环境中，IVF 查询被拆分为多个 rank 上的本地搜索和 root 端的全局合并。与单机多线程共享索引不同，MPI 中每个 rank 拥有独立地址空间，因此必须显式处理数据分片、query 广播、候选收集和 global top-k 合并。

3.4.1 rank 间数据分片

MPI-IVF 将 base 向量划分到不同 rank。每个 rank 只保存本地分片，并在本地构建 IVF 索引。查询时，所有 rank 同时处理同一个 query 或 query batch，各自返回 local top-k。

这种设计可以将单个 rank 的候选扫描规模降低到约原来的 $1/P$。当本地搜索占主要开销时，增加 rank 数可以明显降低查询延迟。但当 rank 数过多时，通信、同步和 root merge 的开销会逐渐占据更大比例，因此 MPI-IVF 存在合适的扩展区间。

3.4.2 query broadcast 与 batch 通信

每个 query 都需要被所有 rank 搜索，因此 root 需要将 query 广播给全部进程。如果逐条 query 广播，会造成大量 MPI 调用和同步开销。为减少该部分开销，可以使用 batch 通信：一次广播多个

query，各 rank 连续处理 batch 内查询，再统一收集候选结果。

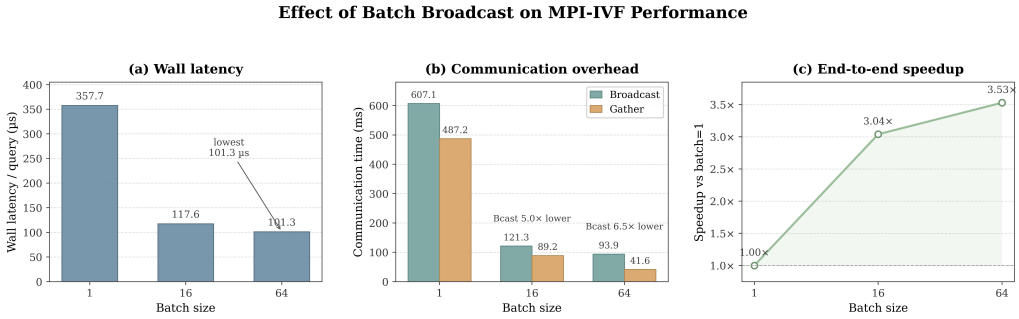


图 5: Batch 通信对 MPI-IVF 查询性能的影响

图 5 表明，batch 通信能够显著减少广播和同步开销。当 batch size 增大后，单位 query 分摊到的通信调用次数减少，wall latency 明显下降。这说明在分布式 ANN 查询中，通信组织方式和本地搜索算法同样重要。

3.4.3 local top-k 搜索

每个 rank 接收到 query 后，在本地 IVF 索引中执行 local search。该过程包括本地 centroid search、候选倒排表扫描和 local top-k 维护。与单机 IVF 不同的是，每个 rank 只处理全局 base 的一个分片，因此 local top-k 只代表局部结果。

local top-k 的作用是减少通信量。如果每个 rank 将所有候选都发送给 root，通信量会随候选规模快速增加；而只返回 top-k，root 需要处理的候选数量为 $P \times k$，规模较小且可控。

3.4.4 global top-k 合并

root 进程收集所有 rank 的 local top-k，并从 $P \times k$ 个候选中选出最终 global top-k。该过程保证了分片搜索能够恢复全局查询结果。

global merge 的开销通常小于本地搜索，但会随 rank 数增加而上升。若 rank 数过多，root 端 gather 和 merge 可能成为新的瓶颈。因此，MPI-IVF 的扩展性并不是线性的，而是取决于本地搜索减少量和通信合并开销之间的平衡。

3.4.5 通信开销与负载均衡

MPI-IVF 的性能由本地搜索、通信和全局合并共同决定。增加 rank 数可以降低本地搜索规模，但也会增加通信和同步成本。数据分片方式还会影响各 rank 的倒排表长度和搜索时间，从而影响负载均衡。

因此，MPI-IVF 的优化重点包括：选择合适 rank 数、使用 batch 通信减少同步、控制每个 rank 的本地候选规模，并避免数据划分造成明显负载不均。总体上，MPI-IVF 更适合搜索规模较大、本地搜索占比较高的场景；当通信占比过高时，继续增加 rank 的收益会下降。

3.5 GPU-IVF 批量查询

GPU-IVF 将 IVF 查询组织为 batch 计算。GPU 适合处理大规模规则化矩阵运算，因此 centroid search 和候选簇内距离计算都可以尽量转化为批量矩阵乘法或批量 kernel 执行。与 CPU 版本相比，

GPU-IVF 的关键不只是减少候选数量，还包括提高 batch 内访问局部性，减少小规模、不规则计算带来的开销。

3.5.1 centroid 搜索

GPU-IVF 首先计算 query batch 与 IVF centroids 的相似度。设 query batch 为 Q，centroid 矩阵为 C，则中心得分可以表示为

$$S_c = QC^T.$$

该计算可以通过 GPU batch GEMM 高效完成。随后，对每个 query 选取 top-*nprobe* 个候选簇，进入下一阶段候选搜索。

centroid search 决定后续访问哪些倒排表，因此直接影响 Recall 和候选扫描规模。*nprobe* 越大，Recall 通常越高，但候选扫描量和合并开销也会增加。

3.5.2 batch 内候选簇搜索

选出候选簇后，GPU-IVF 将访问同一倒排表的 query 组织成子批，并对 query 子批与该簇内 base 向量执行批量距离计算。这种方式可以减少重复启动小 kernel 的开销，并提高矩阵计算吞吐。

但 IVF 的倒排表长度和 query 的候选簇集合并不完全规则。如果同一个 batch 内 query 访问的候选簇差异很大，就会导致 unique probe lists 增多，降低 batch 内访问局部性。因此，GPU-IVF 不仅需要调节 *nprobe*，还需要考虑 query 组织方式。

3.5.3 query grouping 与访问局部性

query grouping 的思想是调整 query 在 batch 中的排列顺序，使同一 batch 内的 query 尽可能访问相似的 IVF list。该操作不改变单个 query 的候选簇集合，因此理论上不改变 Recall，但可以提高 batch 内倒排表访问重合度，减少冗余访问和调度开销。

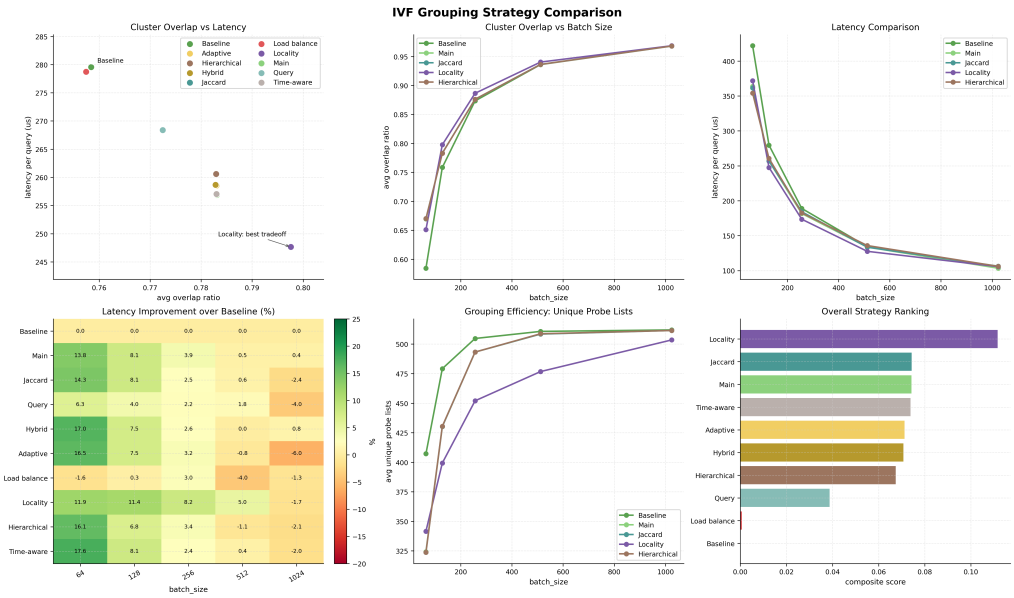


图 6: GPU-IVF 中不同 query grouping 策略的效果对比

图 6 表明，不同 grouping 策略对 Recall 影响很小，但会影响平均延迟和访问局部性。Locality

grouping 通过提高 batch 内 probe list 重合度，减少 unique probe lists，从而降低查询延迟。这说明 GPU-IVF 的瓶颈不仅来自候选数量，也来自 batch 内访问模式是否适合 GPU 执行。

3.6 实验结果与分析

表 3 汇总了 IVF 类方法在不同并行架构下的代表结果。

表 3: IVF 类索引并行化代表结果

方法	并行架构	Recall@10	Latency ($\mu s/query$)	主要意义
IVF-SIMD Pthread	单机多线程	0.959002	81.33	query 级并行
IVF-SIMD OpenMP	单机多线程	0.959002	88.60	调度策略影响性能
MPI-IVF batch	MPI	0.97305	101.296	batch 通信降低同步开销
GPU-IVF ($nprobe = 64$)	CUDA	0.99505	486.25	高 Recall GPU-IVF 配置
GPU-IVF Locality grouping	CUDA	0.94570	247.65	提高 batch 访问局部性

综合来看，IVF 类索引实现了从“全量扫描”到“候选簇扫描”的结构性转变。单机多线程主要利用 query 间独立性提升吞吐；MPI-IVF 通过 rank 间数据分片降低本地搜索规模，但需要处理通信和 global merge；GPU-IVF 则通过 batch GEMM 和 query grouping 发挥 GPU 吞吐优势。

因此，IVF 的并行化不仅是把同一个查询过程简单拆到多个线程或进程中，而是要围绕候选生成、候选访问和结果合并重新组织计算。其核心瓶颈也随架构变化而迁移：在单机上主要是候选扫描和线程调度，在 MPI 中变为通信与合并，在 GPU 中则变为 batch 组织和访问局部性。

4 IVF-PQ 类索引的并行化

IVF-PQ 是 IVF 与 Product Quantization 的组合方法。IVF 通过倒排表减少候选数量，PQ 则通过压缩编码降低候选扫描阶段的距离估计成本。与普通 IVF 直接扫描原始 float 向量不同，IVF-PQ 在粗排阶段主要使用 PQ code 查表累加得到近似距离，再根据需要对部分候选执行 rerank。因此，本节关注的核心问题是：如何在压缩率、近似误差、rerank 成本和并行开销之间取得平衡。

4.1 IVF-PQ 基本原理

IVF-PQ 首先沿用 IVF 的倒排索引结构，将 base 向量划分到 $nlist$ 个倒排表中。查询时，算法先选择与 query 最接近的 $nprobe$ 个倒排表，只在这些倒排表内部进行候选搜索。

与普通 IVF 不同，IVF-PQ 不直接使用原始 float 向量进行候选粗排，而是使用 PQ 压缩编码估计候选距离。设原始向量维度为 d，PQ 将其划分为 M 个子空间：

$$x = [x^{(1)}, x^{(2)}, \ldots, x^{(M)}].$$

每个子空间分别训练一个码本，候选向量 x 被编码为

$$code(x) = (a_1, a_2, \ldots, a_M),$$

其中 a_m 表示第 m 个子空间中最接近的 codeword 编号。

因此，IVF-PQ 查询流程可以概括为：

1. 使用 IVF centroid search 选出 *nprobe* 个候选倒排表；
2. 为当前 query 构建 PQ lookup table；
3. 在候选倒排表中扫描 PQ code，并通过查表累加得到近似距离；
4. 保留近似得分最高的 top-p 个候选；
5. 对 top-p 候选使用原始 float 向量 rerank，输出最终 top-k。

其中，*nprobe* 控制 IVF 的搜索范围，p 或 $rerank_p$ 控制进入精排阶段的候选数量。前者影响候选覆盖率，后者影响 PQ 近似误差能否被精排修正。

4.2 PQ 码本训练与向量编码

PQ 的训练过程在每个子空间中独立进行。对于第 m 个子空间，训练得到 K_s 个聚类中心：

$$C_m = \{c_m^1, c_m^2, \ldots, c_m^{K_s}\}.$$

对于 base 向量 x_i 的第 m 个子向量 $x_i^{(m)}$，其 PQ code 为

$$a_m = \arg\min_t dist(x_i^{(m)}, c_m^t).$$

编码后，每个 base 向量只需要保存 M 个 codeword 编号，而不是完整的 d 维 float32 向量。以 96 维 float32 向量为例，原始向量需要

$$96 \times 4 = 384 \text{ bytes}.$$

若设置 $M = 8$，每个向量只保存 8 个 uint8 code，压缩率约为 48×；若设置 $M = 12$，压缩率约为 32×。这种压缩能够显著降低存储开销和候选扫描时的访存压力，但也会引入量化误差。

4.3 PQ 查表粗排与 rerank

对于给定 query q，IVF-PQ 首先为每个子空间构建 lookup table：

$$LUT_m[t] = dist(q^{(m)}, c_m^t).$$

随后，对于某个候选向量 x_i 的 PQ code $(a_1, \ldots, a_M)$，其近似距离为

$$\hat{d}(q, x_i) = \sum_{m=1}^{M} LUT_m[a_m].$$

查表粗排的优势是计算量较低：候选距离计算不再需要访问完整 float 向量，而是变成 M 次表项读取和累加。但 PQ 近似距离并不完全等价于原始空间距离，真实近邻可能因为量化误差在粗排中被排到较低位置。因此，本文在 CPU 和分布式版本中使用 rerank：先用 PQ 近似距离保留 top-p 候选，再对这些候选使用原始 float 向量重新计算精确距离。

rerank 的作用是修正 PQ 粗排误差，但也会重新引入 float 距离计算开销。因此，$rerank_p$ 是 IVF-PQ 中最关键的参数之一。较小的 $rerank_p$ 延迟低，但 Recall 容易下降；较大的 $rerank_p$ 可以恢复召回率，但会削弱 PQ 带来的速度优势。

4.4 单机多线程 IVF-PQ

在单机多线程环境中，IVF-PQ 的并行化主要采用 query-level parallelism。不同线程处理不同 query，每个 query 独立完成 centroid search、PQ LUT 构建、PQ 粗排、rerank 和 top-k 输出。由于索引结构在查询阶段只读，因此多线程之间几乎不需要同步。

4.4.1 PQ LUT 构建并行

PQ LUT 构建是每个 query 独立执行的阶段。不同 query 的 lookup table 不共享，因此不同线程可以独立构建各自的 LUT，不需要加锁。

LUT 构建本身的计算规模通常小于候选扫描和 rerank，但当 query 数量较多时，该阶段会被重复执行，仍会影响端到端 latency。因此，在实现中可以继续使用 SIMD 加速 LUT 构建中涉及的子空间距离计算。

4.4.2 候选粗排并行

候选粗排阶段扫描被选中倒排表中的 PQ code。每个候选的近似距离由查表累加得到，计算较轻，但访存模式比连续 float 向量扫描更复杂。PQ code 更短，可以降低存储带宽压力；但查表访问和候选堆维护会成为新的开销来源。

在 query-level 并行下，候选粗排仍然属于单个 query 的内部流程，不同线程之间不共享候选堆。因此，相同参数下，多线程版本通常不会改变 Recall，主要影响 wall latency。

4.4.3 rerank 精排阶段

rerank 阶段对 PQ 粗排得到的 top-p 候选重新计算原始 float 距离。该阶段的开销与 $rerank_p$ 成正相关：p 越大，Recall 越高，但精排成本越高。由于 rerank 会重新访问原始 float 向量，因此可以继续使用 SIMD 优化。

代表性结果表明，IVF-PQ-SIMD 在单机多线程场景中可以取得较好的 Recall-Latency 平衡。例如 OpenMP 版本 Recall@10 为 0.998850，wall latency 为 19.59 μs；Pthread 版本 Recall@10 同样为 0.998850，wall latency 为 20.59 μs。这说明在合适的 $nprobe$ 和 $rerank_p$ 下，IVF-PQ 可以同时保持较高召回率和较低查询延迟。

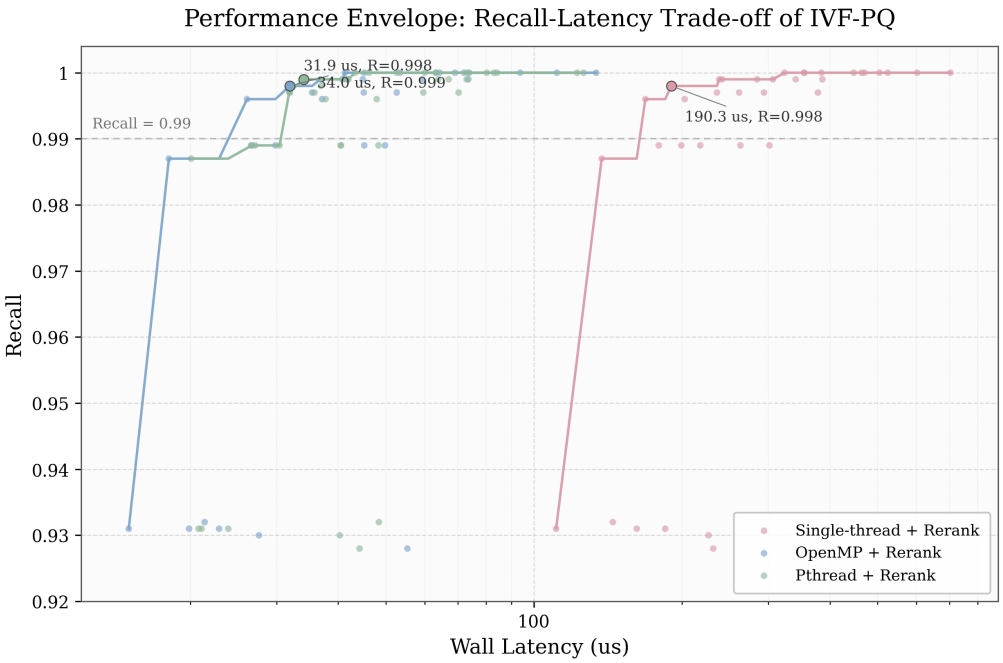


图 7: IVF-PQ-SIMD 的 Recall-Latency 包络

4.5 MPI-IVF-PQ 分布式并行

在分布式环境中，IVF-PQ 可以与数据分片结合。每个 rank 保存本地 base 分片，并构建本地 IVF-PQ 索引。查询阶段，各 rank 独立执行本地 PQ 粗排和 rerank，最后由 root 进程完成 global top-k 合并。

4.5.1 rank 本地 PQ 编码

每个 rank 只对本地 base 分片进行 PQ 编码。这样可以避免所有 base 向量集中到单个进程进行编码，也避免构建完成后再分发完整 PQ code。每个 rank 的本地索引包含本地 IVF 倒排表、PQ code、必要的原始 float 向量以及 PQ 码本信息。

这种设计降低了单个 rank 的存储压力，也使得查询阶段可以天然并行。但本地索引只能覆盖全局数据的一部分，因此每个 rank 的结果都只是 local top-k，必须经过全局合并才能得到最终答案。

4.5.2 local approximate search

查询阶段，每个 rank 接收 query 后，先选择本地 IVF 中的 *nprobe* 个候选倒排表，然后构建 PQ LUT，并对候选表中的 PQ code 执行近似距离计算。该阶段输出的是进入 rerank 的本地候选集合，而不是最终结果。

与直接计算 float 距离相比，PQ 粗排降低了本地候选扫描成本。但如果 *nprobe* 或 *rerank_p* 过小，真实近邻可能无法进入本地 rerank 阶段，最终 Recall 会下降。

4.5.3 rerank 与 global merge

在每个 rank 内部，PQ 粗排得到 top-p 候选后，会使用原始 float 向量进行 SIMD rerank，得到 local top-k。随后，root 进程收集所有 rank 的 local top-k，并执行 global top-k merge。

这里的 rerank 和 global merge 分别解决两个问题：rerank 用于修正 PQ 近似距离误差，global merge 用于修正数据分片带来的局部性。由于每个 rank 只返回少量候选，通信量相对可控；但最终 Recall 依赖于各 rank 的本地粗排是否保留了足够好的候选。

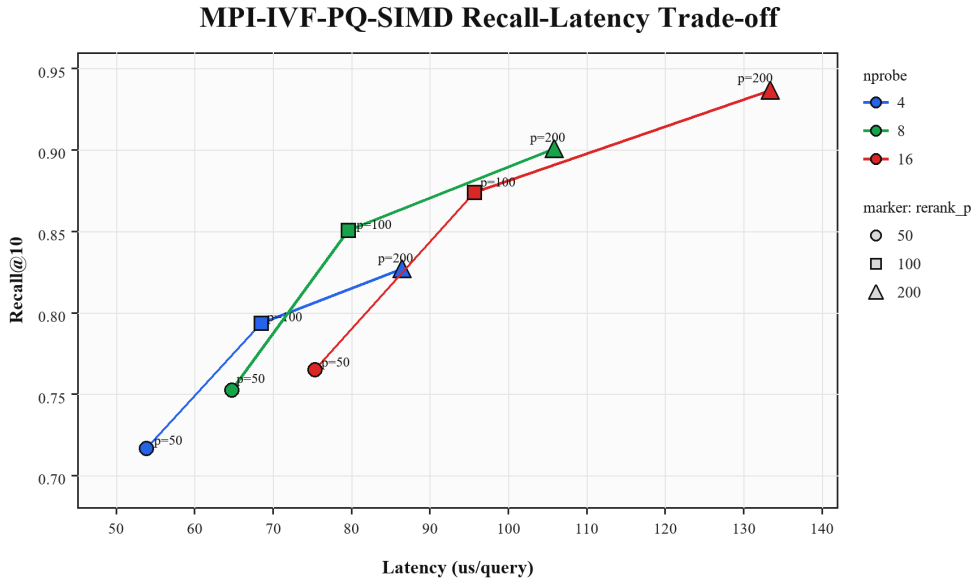


图 8: 分布式 IVF-PQ 的 Recall-Latency trade-off

图 8 以 latency 为横轴、Recall@10 为纵轴展示分布式 IVF-PQ 的权衡关系。图中每条折线固定一个 *nprobe*，点标记表示不同的 *rerank_p*。可以看到，在相同 *nprobe* 下，增大 *rerank_p* 会使更多候选进入精排阶段，因此 Recall 提升，但 latency 也同步增加；在相同 *rerank_p* 下，增大 *nprobe* 会扩大倒排表搜索范围，通常进一步提升 Recall，但也带来额外候选扫描开销。最高配置达到 Recall@10 为 0.93660，latency 为 133.399 μs。

4.6 GPU IVF-PQ-lite

GPU IVF-PQ-lite 是 IVF-PQ 在 GPU 上的轻量化实现。它的重点是验证 PQ 压缩表示在 GPU-IVF 框架下的可行性和存储收益。与 CPU 和分布式版本不同，GPU lite 版本没有加入 residual quantization 或 reranking，因此其 Recall 明显低于 full-vector IVF。

4.6.1 压缩表示与存储开销

GPU IVF-PQ-lite 将 base 向量编码为 PQ code。对于 96 维 float32 向量，原始存储为 384 bytes。若 $pq_m = 8$，每个向量只需 8 bytes，压缩率为 48×；若 $pq_m = 12$，每个向量需要 12 bytes，压缩率为 32×。

这种压缩对于 GPU 场景具有实际意义：一方面可以减少显存占用，使更大规模的向量库能够放入 GPU；另一方面可以降低候选扫描阶段的全局内存访问量。但压缩率越高，子空间表达能力越弱，近似距离误差也越大。

4.6.2 GPU 查询流程

GPU IVF-PQ-lite 的查询流程与 GPU-IVF 类似。首先，使用 GPU 计算 query batch 与 centroids 的相似度，并选择 *nprobe* 个候选簇；随后为 query 构建 PQ lookup table；最后在候选簇中扫描 PQ code，通过查表累加得到近似得分并维护 Top-k。

由于没有 rerank，最终结果完全依赖 PQ 近似距离。这使得查询流程更轻量，也更能体现压缩表示本身的影响，但同时会带来较明显的 Recall 损失。

4.6.3 Recall 损失来源

GPU IVF-PQ-lite 的 Recall 损失主要来自三个方面。第一，PQ 将原始 float 向量映射为子空间 code，近似距离本身存在量化误差。第二，轻量版实现没有使用 residual quantization，无法充分建模 IVF 簇内残差分布。第三，未加入 rerank，真实近邻一旦在 PQ 粗排中排序靠后，就无法通过精排阶段被恢复。

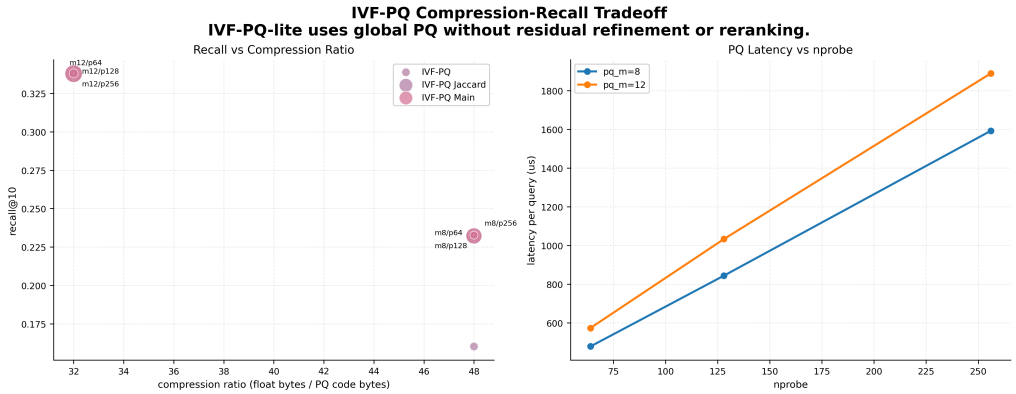


图 9: GPU IVF-PQ-lite 的压缩率与召回率权衡

图 9 展示了 GPU IVF-PQ-lite 的压缩率、Recall 和 latency 之间的关系。当 $pq_m = 8$、$pq_ksub = 256$、$nprobe = 64$ 时，压缩率为 48×，Recall@10 约为 0.2327，latency 约为 466.02 μs；当 $pq_m = 12$ 时，压缩率降低为 32×，Recall@10 提升到 0.33805，latency 约为 576.30 μs。这说明增加子空间数量可以提升 PQ 表达能力，但会牺牲压缩率并增加一定查询开销。

4.7 实验结果与分析

表 4 汇总了 IVF-PQ 类索引在不同并行架构下的代表结果。

表 4: IVF-PQ 类索引并行化代表结果

方法	并行架构	Recall@10	Latency ($\mu s/query$)	主要定位
IVF-PQ-SIMD OpenMP	单机多线程	0.998850	19.59	高 Recall、低延迟
IVF-PQ-SIMD Pthread	单机多线程	0.998850	20.59	与 OpenMP 接近
MPI-IVF-PQ-SIMD	MPI	0.93660	133.399	分布式压缩搜索
GPU IVF-PQ-lite ($pq_m = 8$)	CUDA	0.23270	466.02	48× 压缩
GPU IVF-PQ-lite ($pq_m = 12$)	CUDA	0.33805	576.30	32× 压缩

综合来看，IVF-PQ 的优势在于同时减少候选数量和单个候选的距离估计成本。单机多线程版本

依靠 query-level parallelism、PQ 查表粗排和 SIMD rerank，在高 Recall 区间取得较低查询延迟；分布式版本进一步结合 rank 间数据分片，但需要处理本地近似搜索和全局合并之间的关系；GPU lite 版本则突出 PQ 的存储压缩能力，但由于缺少 residual quantization 和 rerank，Recall 明显低于 full-vector IVF。

因此，IVF-PQ 不是简单的“更快 IVF”，而是一类需要围绕压缩率、候选覆盖、rerank 成本和并行架构共同设计的索引方法。在需要高 Recall 的场景中，rerank 是恢复精度的关键；在存储或显存受限的场景中，PQ 压缩具有价值，但必须谨慎处理压缩带来的召回损失。

5 HNSW 类图索引的并行化

HNSW（Hierarchical Navigable Small World）是一类基于图结构的近似最近邻索引。与 IVF 通过聚类缩小候选范围、IVF-PQ 通过压缩编码降低候选距离估计成本不同，HNSW 将 base 向量组织为近邻图，并在查询时沿图边进行导航式搜索。它的优势在于：索引构建完成后，单次查询通常只需要访问少量节点，就能获得较高 Recall。因此，HNSW 是本文中高召回、低延迟场景下的重要图索引方法。

HNSW 的并行化方式也具有自身特点。单个 query 的图搜索路径依赖当前候选队列和访问状态，不适合简单拆分为多个线程并行执行。因此，在单机多线程中，本文主要采用多 query 并行；在分布式场景中，则采用 shard-HNSW，即每个 rank 构建本地图索引，各自完成 local search 后再进行全局候选合并。

5.1 HNSW 基本原理

HNSW 通过多层小世界图组织向量。高层图节点较少、连接较稀疏，用于快速定位搜索方向；底层图节点最多、连接更密集，用于进行精细搜索。每个节点表示一个 base 向量，边表示近邻关系。

查询时，算法从入口点开始，在高层图中采用贪心策略，不断移动到距离 query 更近的节点；当无法继续改进时，进入下一层，并以上一层得到的节点作为新的入口点。最终在底层图中维护候选队列，通过扩展候选邻居得到最终 top-k 结果。整体流程可以表示为

$$q \rightarrow \textit{entry point} \rightarrow \textit{upper layer greedy search} \rightarrow \textit{bottom layer candidate expansion} \rightarrow \textit{top-}k.$$

HNSW 的主要参数包括：

1. M：每个节点保留的邻居数量，影响图的稠密程度；
2. $efConstruction$：索引构建时的候选队列大小，影响图质量和构建成本；
3. $efSearch$：查询阶段的候选队列大小，直接影响 Recall 与 latency。

其中，$efSearch$ 是查询阶段最关键的参数。较大的 $efSearch$ 会扩大搜索范围，使 Recall 更稳定，但也会增加图遍历和候选队列维护开销。

5.2 单机多线程 HNSW

在单机环境中，HNSW 的多线程并行主要采用 query-level parallelism。HNSW 索引在查询阶段是只读结构，因此多个线程可以共享同一张图，而每个线程独立维护自己的候选队列、访问集合和 top-k 结果。

5.2.1 多 query 并行

多 query 并行的核心思想是：不同线程处理不同 query，而不是拆分单个 query 内部的图搜索过程。这样做的原因是，HNSW 单个 query 的搜索过程存在较强数据依赖，下一步访问哪个节点取决于当前候选队列和已经访问过的节点集合。如果强行对单个 query 做细粒度并行，通常需要复杂的同步和候选队列合并，反而可能抵消收益。

因此，HNSW 更适合在 query 维度并行。该方式不会改变单个 query 的搜索逻辑，在相同索引参数和 $efSearch$ 下，Recall 与串行版本保持一致，主要提升 batch query 的整体吞吐。

代表性结果中，HNSW OpenMP 版本 Recall@10 达到 1.000000，wall latency 为 15.77 μs；Pthread 版本 Recall@10 同样为 1.000000，wall latency 为 22.09 μs。这说明 HNSW 在索引构建完成后，查询阶段能够以较少节点访问获得很高召回率。

5.2.2 efSearch 对查询性能的影响

$efSearch$ 控制查询阶段候选队列的大小。较小的 $efSearch$ 可以减少图遍历和候选维护开销，但可能导致搜索提前停止；较大的 $efSearch$ 可以访问更多节点，使 Recall 更稳定，但 latency 也随之上升。

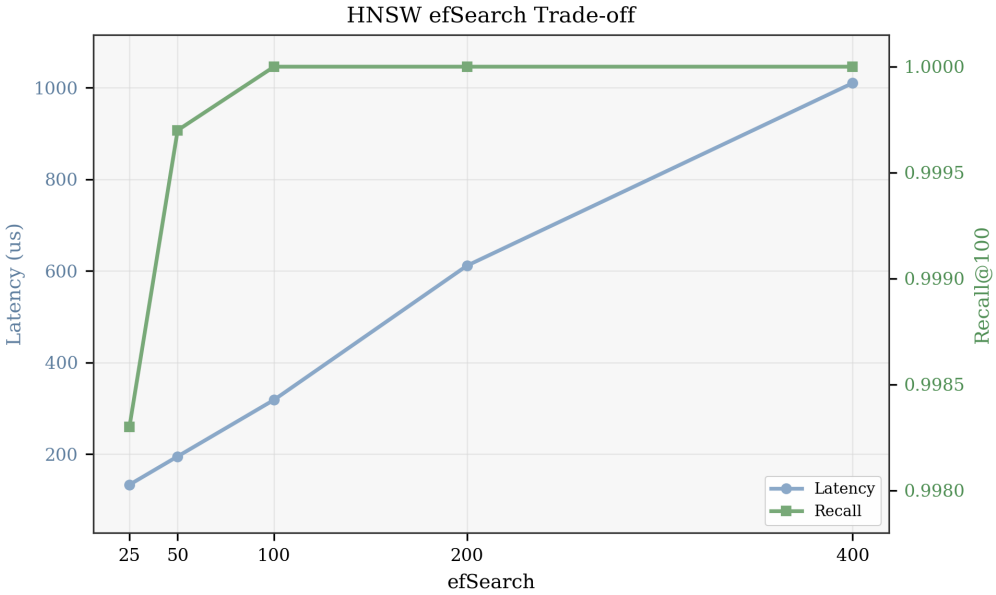


图 10: HNSW 在不同 $efSearch$ 下的 Recall-Latency 权衡

从图 10 可以看出，随着 $efSearch$ 增大，HNSW 的查询延迟明显上升；但在本实验数据规模下，Recall 在较小 $efSearch$ 时已经接近 1.0，继续增大 $efSearch$ 的边际收益有限。因此，HNSW 的参数选择不应盲目增大搜索宽度，而应在满足 Recall 要求后选择较小的 $efSearch$ 以降低延迟。

5.3 MPI shard-HNSW

在分布式环境中，直接维护跨 rank 的全局 HNSW 图较为复杂，因为图边可能连接到其他进程上的节点，查询过程中会产生频繁远程访问。为避免这一问题，本文采用 shard-HNSW：先将 base 向量分片，每个 rank 在本地分片上独立构建 HNSW 图，查询时各 rank 独立搜索本地图，最后由 root 进行全局合并。

5.3.1 数据分片与本地图构建

设 MPI 进程数为 P，base 集合被划分为

$$X = X_0 \cup X_1 \cup \cdots \cup X_{P-1}.$$

每个 rank r 只使用本地分片 X_r 构建 HNSW 图 G_r。这种设计避免了跨进程维护图边，也使得每个 rank 的索引构建和查询过程可以独立完成。

这种方案的代价是，每个本地图只覆盖全局数据的一部分。局部 HNSW 图并不等价于完整数据上的全局 HNSW 图，因此单个 rank 的 local top-k 只代表本地分片结果。为了恢复全局搜索结果，需要所有 rank 同时参与查询，并将候选结果汇总。

5.3.2 local HNSW 搜索

查询阶段，root 将 query 广播给所有 rank。每个 rank 在本地 HNSW 图 G_r 上执行标准 HNSW 搜索，并返回 local top-k。由于各 rank 的本地搜索互相独立，计算部分具有较好的并行性。

shard-HNSW 的延迟受两方面影响。一方面，rank 数增加后，每个 rank 的本地图规模变小，本地搜索更快；另一方面，rank 数增加也会带来更多 broadcast、gather 和 merge 开销。此外，数据分片方式会影响本地图对全局近邻结构的覆盖质量。

5.3.3 全局候选合并

root 进程收集所有 rank 的 local top-k，并从 $P \times k$ 个候选中选出 global top-k。该过程与分布式 IVF 的 global merge 类似，但 local search 的来源从倒排表扫描变成了图搜索。

由于每个 rank 只返回少量候选，合并规模通常可控。但如果某个真实近邻没有进入任何 rank 的 local top-k，全局合并也无法恢复。因此，shard-HNSW 的 Recall 取决于本地图质量、$efSearch$、分片方式以及 local top-k 候选是否足够覆盖真实近邻。

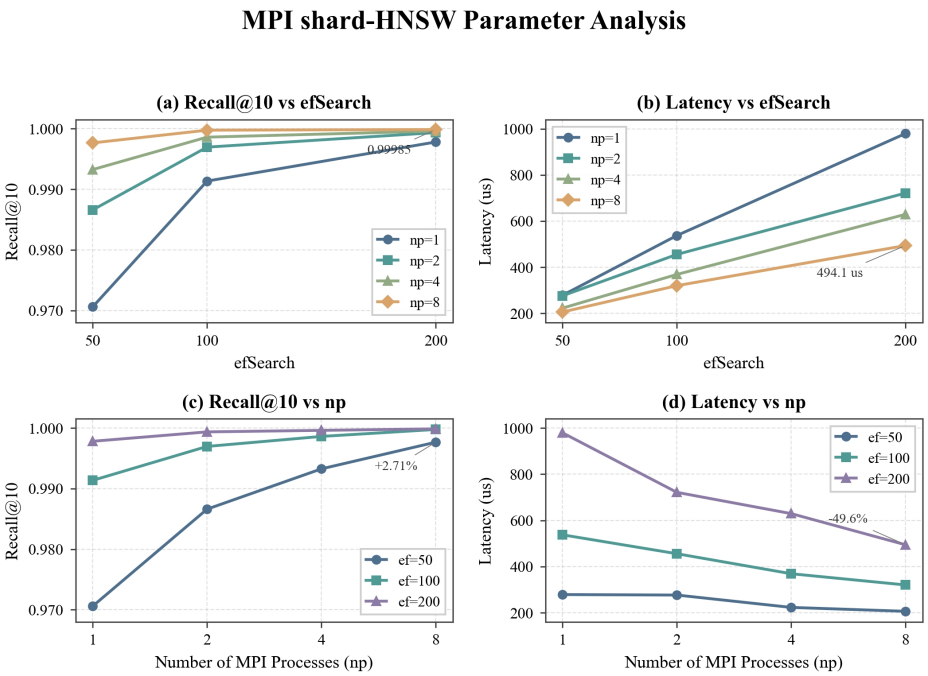


图 11: shard-HNSW 在不同 $efSearch$ 与进程数下的性能分析

图 11 表明，增大 $efSearch$ 可以提升 Recall，但会带来更高 latency；增加进程数可以降低每个 rank 的本地搜索规模，从而减少查询延迟。代表性 shard-HNSW 配置可以达到 Recall@10 约 0.99765、latency 约 205.815 μs，在高 Recall 区域具有较好的表现。

5.4 IVF-HNSW 混合索引

IVF-HNSW 试图结合 IVF 的粗粒度候选过滤能力和 HNSW 的局部图搜索能力。其基本思想是：先用 IVF 将 base 向量划分到不同 cluster 中，再在每个 cluster 内部构建 HNSW 图。查询时，先选择若干相关 cluster，再在对应 cluster 内执行 HNSW 搜索。

5.4.1 簇内 HNSW 构建

IVF-HNSW 首先进行聚类，得到 $nlist$ 个 cluster。对于每个 cluster，使用其中的 base 向量单独构建一个局部 HNSW 图。查询时，算法先根据 query 与聚类中心的距离选择 top-$nprobe$ 个 cluster，然后进入这些 cluster 内部的 HNSW 图搜索。

其流程可以表示为

$$q \to \mathrm{Top}_{nprobe}(\{c_j\}) \to \{G_j\}_{j\in S} \to \textit{top-}k,$$

其中 $\{c_j\}$ 为 IVF 聚类中心，S 为候选 cluster 集合，G_j 为第 j 个 cluster 内部的 HNSW 图。

这种设计的好处是每个 HNSW 图规模更小，图搜索只在候选 cluster 内进行；但它也引入了新的风险：如果真实近邻所在 cluster 没有被 $nprobe$ 选中，后续 HNSW 搜索无法弥补这一候选生成错误。

5.4.2 nprobe 与 efSearch 的共同影响

IVF-HNSW 同时受到 $nprobe$ 和 $efSearch$ 控制。$nprobe$ 决定访问多少个 cluster，主要影响候选覆盖率；$efSearch$ 决定每个 cluster 内 HNSW 搜索的深度，主要影响局部图搜索质量。

实验结果表明，$nprobe$ 对 Recall 的影响通常更明显，因为它决定真实近邻所在区域是否被访问；$efSearch$ 增大则主要增加局部图搜索深度和延迟。在代表性配置 $nprobe = 16, efSearch = 200$ 下，Recall@10 约为 0.95895，latency 约为 414.761 μs。这说明 IVF-HNSW 并不一定优于 shard-HNSW，它更适合用于分析倒排过滤与图搜索结合后的参数影响。

5.5 centroid-HNSW + IVF-HNSW

在 IVF-HNSW 的基础上，还可以进一步在 IVF 聚类中心上建立 centroid-HNSW，用图搜索代替暴力扫描所有 centroid。此时 query 不再直接计算与全部 $nlist$ 个中心的距离，而是先在 centroid-HNSW 图上搜索候选 cluster，再进入 cluster 内部的 HNSW 图。

其流程可以表示为

$$q \to G_{centroid} \to \{G_j\}_{j\in S} \to \textit{top-}k.$$

该方法的动机是降低 centroid search 的开销。但实验结果表明，在当前数据规模和 $nlist$ 设置下，直接扫描聚类中心的成本并不高；额外引入 centroid-HNSW 后，反而增加了图搜索和候选维护开销。

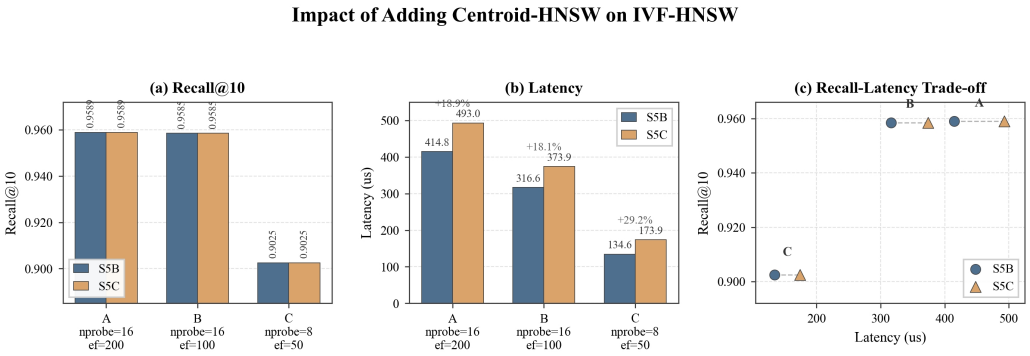


图 12: 引入 centroid-HNSW 对 IVF-HNSW 查询性能的影响

从图 12 可以看出，centroid-HNSW 基本没有提升 Recall，反而增加了 latency。例如在高精度配置下，latency 从约 414.8 μs 增加到约 493.0 μs。这说明层级结构并不是越复杂越好，当上层搜索空间本身不大时，额外图索引可能形成负优化。

5.6 实验结果与分析

表 5 汇总了 HNSW 类图索引在不同并行方式下的代表结果。

表 5: HNSW 类图索引并行化代表结果

方法	并行架构	Recall@10	Latency ($\mu s/query$)	主要定位
HNSW OpenMP	单机多线程	1.000000	15.77	高 Recall、低延迟
HNSW Pthread	单机多线程	1.000000	22.09	query 级并行
shard-HNSW	MPI	0.99765	205.815	分片图索引
IVF-HNSW	MPI	0.95895	414.761	倒排过滤 + 簇内图搜索
centroid-HNSW + IVF-HNSW	MPI	约 0.959	约 493.0	当前规模下负优化

综合来看，HNSW 类图索引的优势在于查询阶段能够通过图导航访问较少节点，从而在高 Recall 下取得较低 latency。单机多线程场景中，HNSW 的 query-level parallelism 简单有效；分布式场景中，shard-HNSW 通过本地图搜索和全局候选合并，在高 Recall 区域保持了较好表现。

但 HNSW 也有自身代价。首先，HNSW 的索引构建成本和内存开销通常高于 IVF；其次，单个 query 的图搜索过程具有较强数据依赖，不适合简单进行细粒度并行；最后，在分布式环境中，本地图会破坏完整全局图结构，需要通过多 rank 搜索和 global merge 恢复结果质量。

IVF-HNSW 和 centroid-HNSW + IVF-HNSW 的结果进一步说明，组合索引并不必然优于单一索引。IVF-HNSW 引入了 $nprobe$ 和 $efSearch$ 两个参数，调参空间更大；centroid-HNSW 虽然试图减少中心搜索开销，但在当前规模下额外图搜索成本超过收益。因此，HNSW 更适合作为高 Recall、低查询延迟的强基线，而其混合索引版本需要结合数据规模、构建成本和查询开销谨慎选择。

6 跨方法综合讨论

前文分别讨论了无索引扫描、量化方法、IVF、IVF-PQ 和 HNSW 等 ANN 查询方法，以及它们在 SIMD、多线程、MPI 和 CUDA 架构下的并行化方式。本节进一步从整体角度比较这些方法的

Recall-Latency 特征，分析性能瓶颈如何随算法结构和并行架构变化而迁移，并总结不同场景下的方法选择。

6.1 不同索引的 Recall-Latency 特征

不同 ANN 方法在 Recall-Latency 空间中的位置，主要由两类因素决定：一是候选生成方式，即算法访问多少候选向量；二是距离估计方式，即算法使用原始 float 距离、量化近似距离还是图搜索路径来判断近邻。

无索引 Flat Scan 访问全部 base 向量，因此 Recall 最高，但 latency 最大。Flat-SIMD 在不改变搜索结果的前提下加速距离计算，因此可以降低延迟但无法改变全量扫描的复杂度。SQ 和 PQ 通过低位宽表示或查表近似降低粗排成本，但需要 rerank 控制召回损失。IVF 通过倒排表减少候选数量，能够在 Recall 和 latency 之间形成更灵活的折中。IVF-PQ 进一步压缩候选表示，在候选数量和单个候选计算成本两方面同时优化，但其精度依赖 rerank。HNSW 则通过图导航访问少量节点，在高 Recall 区域通常具有较低查询延迟，但需要更高的索引构建和内存成本。

表 6: 不同 ANN 方法的 Recall-Latency 特征

方法类别	候选生成方式	Recall-Latency 特征	主要代价
Flat Scan	全量扫描全部 base 向量	Recall 最高，latency 最大	距离计算量大
Flat-SIMD	全量扫描 + SIMD 距离计算	Recall 不变，latency 降低	仍需访问全部向量
SQ / PQ	量化粗排 + rerank	可降低粗排成本，Recall 依赖候选数	量化误差与 rerank 成本
IVF	centroid 选择 + 倒排表扫描	通过 *nprobe* 调节 Recall 与 latency	参数敏感、倒排表不均
IVF-PQ	IVF 候选 + PQ 查表粗排	候选数和距离估计成本同时下降	PQ 误差与 rerank 成本
HNSW	图导航式搜索	高 Recall 下 latency 较低	构建成本和内存开销较高

从代表性结果看，无索引扫描方法的延迟通常处于较高区间；量化方法能够将延迟明显降低，但需要控制 Recall 损失；IVF 和 IVF-PQ 通过索引结构减少候选访问，是更适合大规模查询的折中方法；HNSW 在查询阶段表现突出，适合高 Recall、低延迟场景。需要注意的是，本文涉及多个硬件平台，因此不同方法的绝对 latency 不应被简单理解为完全公平的硬件横向比较，更应关注其方法特征和瓶颈变化趋势。

6.2 计算、访存与通信瓶颈迁移

ANN 查询的主要瓶颈并不是固定不变的。随着方法从无索引扫描发展到量化、倒排索引、压缩索引和图索引，瓶颈会从计算逐步迁移到访存、候选维护、通信合并和 GPU 数据搬运。

在 Flat Scan 阶段，主要瓶颈是高维 float 内积计算。此时 SIMD、循环展开和预取等优化能够直接降低距离计算成本。进入 SQ/PQ 后，单次距离计算变得更轻，但量化误差、查表访问和 rerank 候选数成为新的关键因素。进入 IVF 后，候选数量减少，但倒排表访问、*nprobe* 参数选择和候选 top-k 维护成为主要开销。进入 MPI 分布式场景后，本地搜索规模下降，但 query broadcast、candidate gather 和 global merge 变得重要。进入 GPU 场景后，距离计算可以由 batch GEMM 高效完成，但数据搬运、GPU Top-K、batch 组织和访问局部性会成为新的瓶颈。

表 7: ANN 查询中的瓶颈迁移

阶段	代表方法	主要瓶颈	优化方向
全量扫描	Flat / Flat-SIMD	float 内积计算量大	SIMD、循环展开、prefetch
量化粗排	SQ / PQ	量化误差、查表访问、rerank	corrected score、LUT、候选精排
倒排索引	IVF	候选访问、倒排表长度不均	调节 $nlist$、$nprobe$
压缩索引	IVF-PQ	PQ 误差与 rerank 成本	调节 $nprobe$、$rerank_p$
图索引	HNSW	图遍历和候选队列维护	调节 $efSearch$
分布式	MPI-IVF / shard-HNSW	broadcast、gather、global merge	batch 通信、数据分片、负载均衡
GPU 查询	GPU Flat / GPU-IVF	数据搬运、Top-K、batch 局部性	GPU Top-K、query grouping

因此，ANN 优化不能只盯住某一个固定环节。一个方法在早期瓶颈下有效，可能在新的架构下不再是主要矛盾。例如 SIMD 能显著加速 Flat Scan，但无法减少候选数量；MPI 能降低本地搜索规模，但会引入通信合并开销；GPU 能提高矩阵计算吞吐，但如果 Top-K 和数据搬运仍在 CPU 侧，就会形成新的性能瓶颈。

6.3 SIMD、多核、MPI 与 CUDA 的作用差异

SIMD、多核、MPI 和 CUDA 都属于并行优化，但它们作用在 ANN 查询流程中的层次不同。

SIMD 主要作用于单次距离计算。它不改变候选集合，也通常不改变 Recall，而是降低内积、rerank 或 LUT 构建中的计算成本。多线程主要作用于 query-level parallelism，不同线程处理不同 query，适合查询之间相互独立的 ANN 任务。MPI 主要作用于数据分片，每个 rank 维护本地数据和本地索引，通过 local search 和 global merge 完成分布式查询。CUDA 则主要作用于 batch 计算，将大规模距离计算和 Top-K 选择尽量放到 GPU 上完成。

表 8: 不同并行架构在 ANN 查询中的作用差异

并行方式	作用层次	优势	主要限制
SIMD	单次距离计算 / rerank	不改变结果，直接降低计算开销	无法减少候选数量
多线程	多 query 并行	实现自然，Recall 基本不变	受任务粒度和负载均衡影响
MPI	base 数据分片	降低单 rank 搜索规模，可扩展到多节点	通信、同步和 global merge 开销
CUDA	batch 距离计算与 Top-K	高吞吐，适合 GEMM 和大批量查询	不规则访存、数据搬运和 Top-K 可能成为瓶颈

从整体上看，SIMD 是最底层的计算加速手段，多线程提高单机吞吐，MPI 扩展数据规模，CUDA 提供大规模批量计算能力。这些并行方式不是互斥关系，而是可以组合使用。例如 IVF 查询中，rank 间可以用 MPI 做数据分片，rank 内可以用多线程处理 query，距离计算仍可使用 SIMD；在 GPU-IVF 中，centroid search 和候选簇搜索则可以转化为批量矩阵计算。

6.4 不同应用场景下的方法选择

不同 ANN 方法没有绝对最优，实际选择取决于 Recall 要求、延迟约束、数据规模、内存限制和硬件条件。

如果任务要求最高准确率，且可以接受较高延迟，Flat Scan 或 GPU Flat Top-K 是最稳妥的选择。若希望在高 Recall 下获得较低查询延迟，HNSW 和 IVF-PQ 是更有竞争力的方案。若数据规模较

大，需要多节点扩展，则 MPI-IVF 和 shard-HNSW 更适合。若存储或显存受限，则可以考虑 IVF-PQ 或 GPU IVF-PQ-lite，但必须注意 PQ 压缩带来的 Recall 损失。若查询天然以 batch 形式到达，GPU Flat、GPU-IVF 和 query grouping 能更好发挥 GPU 吞吐优势。

表 9: 不同应用场景下的方法选择

应用需求	推荐方法	原因
最高 Recall，延迟要求较宽松	Flat Scan / GPU Flat Top-K	精确或近似无损，结果稳定
高 Recall，低查询延迟	HNSW / IVF-PQ	图导航或压缩粗排 + rerank 效果较好
需要显式调节 Recall-Latency	IVF / IVF-PQ	$nprobe$、$rerank_p$ 参数含义清晰
多节点扩展	MPI-IVF / shard-HNSW	base 分片降低本地搜索规模
存储或显存受限	IVF-PQ / GPU IVF-PQ-lite	PQ code 显著压缩向量表示
大 batch GPU 查询	GPU Flat / GPU-IVF	batch GEMM、GPU Top-K 和 query grouping 可发挥吞吐优势

因此，方法选择应以应用约束为中心。如果任务更关注准确率，应优先选择高 Recall 方法并接受一定构建或存储成本；如果任务更关注吞吐和延迟，则需要控制候选数量和通信开销；如果任务更关注内存占用，则需要接受 PQ 压缩带来的精度损失，并通过 rerank 或更强码本设计进行补偿。

6.5 本次综合实验新增分析总结

本次综合实验的重点不是把前序实现逐项罗列，而是将不同 ANN 方法放入统一框架中重新组织。原始实验材料中，各方法往往围绕某一种并行接口或某一种索引结构展开；而在综合报告中，本文将它们统一解释为候选生成、距离估计、top-k 维护、rerank 和 merge 等 ANN 查询环节的不同设计。

表 10: 原有实验材料与本次综合分析的关系

分析层次	原有材料关注点	综合报告中的新增组织
方法实现	分别实现扫描、量化、倒排、压缩和图索引	统一为候选生成与距离估计方式的递进关系
并行方式	分别测试 SIMD、多线程、MPI、CUDA	统一为计算、吞吐、数据分片和 GPU batch 四类并行层次
性能结果	分别报告单个方法的 Recall 和 latency	统一分析不同方法的 Recall-Latency 特征
参数实验	分别讨论 $nprobe$、$rerank_p$、$efSearch$ 等参数	统一解释为搜索范围、候选覆盖和精排成本的权衡
瓶颈分析	分别分析某个程序的耗时来源	总结计算、访存、通信和 GPU 数据搬运的瓶颈迁移

通过这种组织方式，本文形成了一条完整主线：首先，无索引扫描说明距离计算是 ANN 查询的基础瓶颈；其次，量化方法说明降低单次距离估计成本可以减少延迟，但需要处理近似误差；随后，IVF、IVF-PQ 和 HNSW 说明改变候选生成结构是进一步提升性能的关键；最后，SIMD、多线程、MPI 和 CUDA 说明同一算法结构在不同并行架构下会暴露不同瓶颈。

因此，本次综合实验的核心结论是：ANN 并行优化不是简单叠加更多线程或更强硬件，而是要根据查询流程中当前主导瓶颈选择合适的算法结构和并行方式。只有同时考虑候选规模、距离估计精度、访存模式、通信合并和硬件执行特点，才能在 Recall 与 Latency 之间取得稳定而有效的折中。

7 总结与展望

7.1 实验结论

本文围绕 DEEP100K 数据集上的 ANN 查询任务，系统整理了无索引扫描、量化方法、IVF、IVF-PQ、HNSW 以及 SIMD、多线程、MPI、CUDA 等并行优化策略。综合实验结果，可以得到以下结论：

- **无索引方法的核心瓶颈是距离计算**。Flat Scan 能保持最高 Recall，但延迟较高；SIMD、循环展开和预取能够在不改变结果的情况下降低 float 内积开销。进一步使用 SQ/PQ 粗排可以降低计算量，但需要 rerank 控制召回损失。
- **IVF 类索引通过减少候选数量降低延迟**。IVF 的关键参数是 *nlist* 和 *nprobe*。较大的 *nprobe* 能提升 Recall，但会增加候选扫描量。多线程、MPI 和 GPU 版本本质上都是围绕候选簇搜索、任务划分和结果合并进行优化。
- **IVF-PQ 适合在存储、延迟和召回率之间折中**。PQ 编码显著降低向量存储和候选距离估计成本，但会引入量化误差。对于高 Recall 场景，rerank 是恢复结果质量的关键；缺少 rerank 的轻量版 IVF-PQ 会出现明显召回损失。
- **HNSW 在高 Recall、低 latency 场景下表现突出**。HNSW 通过图导航减少访问节点数量，查询阶段效率较高。但它的索引构建成本和内存开销较大，且单 query 内部存在较强数据依赖，因此更适合多 query 并行或分片式并行。
- **不同并行架构解决的是不同层次的问题**。SIMD 主要加速单次距离计算，多线程主要提高单机 query 吞吐，MPI 通过数据分片降低本地搜索规模，CUDA 则适合批量距离计算和 GPU Top-K。并行方式的选择应服务于当前主导瓶颈。

总体而言，ANN 查询优化不是简单叠加更多并行资源，而是要根据算法阶段选择合适的优化手段：距离计算瓶颈适合 SIMD/GPU，候选规模瓶颈适合 IVF/HNSW，存储瓶颈适合 PQ，分布式扩展瓶颈则需要重点处理通信和 global merge。

7.2 不足与改进方向

本文仍存在一些不足，后续可以从以下方向继续改进：

- **压缩索引仍可进一步优化**。IVF-PQ 可以继续加入 residual quantization、OPQ 或更完善的 rerank 策略，以降低 PQ 量化误差；GPU IVF-PQ-lite 也可以补充 GPU rerank，提高压缩搜索的 Recall。
- **分布式通信与负载均衡仍有优化空间**。MPI 方法后续可以进一步研究非阻塞通信、通信计算重叠、分层 merge 以及更均衡的数据划分策略，减少 broadcast、gather 和 root merge 带来的额外开销。
- **评价指标可以更加完整**。后续除 Recall 和 average latency 外，还可以加入 build time、memory footprint、throughput、tail latency 等指标，从而更接近真实向量检索系统的评价需求。

通过本次综合实验，可以看到 ANN 并行优化的关键在于识别当前阶段的主导瓶颈，并选择与之匹配的算法结构和并行架构。无论是 SIMD、多线程、MPI 还是 CUDA，它们都不是孤立的加速技巧，而是服务于候选生成、距离估计、结果合并和系统吞吐的不同层次优化。